Test Generation

Generate Playwright test code automatically as you interact with the browser.

How It Works

Every action you perform with `playwright-cli` generates corresponding Playwright TypeScript code. This code appears in the output and can be copied directly into your test files.

Example Workflow

```bash
# Start a session
playwright-cli open https://example.com/login

# Take a snapshot to see elements
playwright-cli snapshot
# Output shows: e1 [textbox "Email"], e2 [textbox "Password"], e3 [button "Sign

# Fill form fields - generates code automatically
playwright-cli fill e1 "user@example.com"
# Ran Playwright code:
# await page.getByRole('textbox', { name: 'Email' }).fill('user@example.com');

playwright-cli fill e2 "password123"
# Ran Playwright code:
# await page.getByRole('textbox', { name: 'Password' }).fill('password123');

playwright-cli click e3
# Ran Playwright code:
# await page.getByRole('button', { name: 'Sign In' }).click();
```

Building a Test File

Collect the generated code into a Playwright test:

```typescript
import { test, expect } from '@playwright/test';

test('login flow', async ({ page }) => {
  // Generated code from playwright-cli session:
  await page.goto('https://example.com/login');
  await page.getByRole('textbox', { name: 'Email' }).fill('user@example.com');
  await page.getByRole('textbox', { name: 'Password' }).fill('password123');
  await page.getByRole('button', { name: 'Sign In' }).click();
```

```
  // Add assertions
  await expect(page).toHaveURL(/.*dashboard/);
});
```

Best Practices

1. Use Semantic Locators

The generated code uses role-based locators when possible, which are more resilient:

```
// Generated (good - semantic)
await page.getByRole('button', { name: 'Submit' }).click();

// Avoid (fragile - CSS selectors)
await page.locator('#submit-btn').click();
```

2. Explore Before Recording

Take snapshots to understand the page structure before recording actions:

```
playwright-cli open https://example.com
playwright-cli snapshot
# Review the element structure
playwright-cli click e5
```

3. Add Assertions Manually

Generated code captures actions but not assertions. Add expectations in your test using one of the recommended matchers:

- `toBeVisible()` — element is rendered and visible
- `toHaveText(text)` — element text content matches
- `toHaveValue(value)` / `toBeEmpty()` — input/select value matches
- `toBeChecked()` / `toBeUnchecked()` — checkbox state matches
- `toMatchAriaSnapshot(snapshot)` — page (or locator) matches a partial accessibility snapshot

Use `playwright-cli generate-locator <target>` to produce the locator expression for the assertion, and the snapshot/eval commands to capture the expected value.

When asserting text content, make sure that generated locator does not contain text from the element itself. `getByTestId()` or `getBy-`

`Label()` usually work well with asserting text. When locator is text-based, prefer `toBeVisible()` instead.

Snapshot to be matched does not have to contain all the information - only capture what's necessary for the assertion. You can use regular expressions for unstable values.

```bash
# Get a stable locator for an element ref to use in the assertion
playwright-cli --raw generate-locator e5
# getByRole('button', { name: 'Submit' })

# Capture expected text content for toHaveText
playwright-cli --raw eval "el => el.textContent" e5

# Capture expected input value for toHaveValue/toBeEmpty
playwright-cli --raw eval "el => el.value" e5

# Capture expected aria snapshot for toMatchAriaSnapshot/toBeChecked
# (whole page, or use a ref to scope to a region)
playwright-cli --raw snapshot
playwright-cli --raw snapshot e5
```

```javascript
// Generated action
await page.getByRole('button', { name: 'Submit' }).click();

// Manual assertions using the outputs above:
await expect(page.getByRole('alert', { name: 'Success' })).toBeVisible();
await expect(page.getByTestId('main-header')).toHaveText('Welcome, user');
await expect(page.getByRole('textbox', { name: 'Email' })).toHaveValue('user@exa
await expect(page.getByRole('checkbox', { name: 'Enable notifications' })).toBeC

// toMatchAriaSnapshot on the whole page, finds a matching region
await expect(page).toMatchAriaSnapshot(`
  - heading "Welcome, user"
  - link /\\d+ new messages?/
  - button "Sign out"
`);

// toMatchAriaSnapshot scoped to a region
await expect(page.getByRole('navigation')).toMatchAriaSnapshot(`
  - link "Home"
  - link /\\d+ new messages?/
  - link "Profile"
`);
```